

Implementation of Feature Selective Validation in Python

Ayush Shukla

Christian Doppler Laboratory for EMC Aware Robust Electronic Systems,
Institute of Electronics, TU Graz

This document talks about the implementation of FSV done in python using code snippets.

Implementation follows the algorithm of FSV computation as described in IEEE Standard for Validation of Computational Electromagnetics Computer Modelling and Simulations along with some other literature, referenced towards the end.

Contents:

- About Feature Selective Validation
- Flow of Code and Procedure
- Example
- Further scope of work
- References

About Feature Selective Validation

Validation in Computational Electromagnetics is a formal process that ensures that the model or modelling technique behaves in an expected manner. A simple subtraction of one data set from the second data set will show the differences but is limited as a true indication of overall agreement or similarity.

The FSV is a technique that combines an amplitude-based comparison with a feature-based comparison to give an overall better indication of the agreement. The FSV has been tried to be calibrated to match human ‘expert’ comparison for somewhat subjective decisions, but we attach natural language labels such as ‘excellent’, ‘very good’, ‘good’, ‘poor’, etc. To the comparisons.

The basis of the FSV technique is the decomposition of the results to be compared into two ‘component’ measures and then the recombination of the results to provide a global goodness of fit measure. The components used are the Amplitude Difference Measure (ADM), which compares the amplitudes and ‘trends’ of the two data sets, and the Feature Difference Measure (FDM), which compares the rapidly changing features (as a function of the independent variable). The ADM and FDM are then combined to form a GDM.

A minimum expectation is that the single value of the Global Difference Measure (GDM) will be reported.

Flow of Code and Procedure

- Prerequisites

Two separate data sets available for comparison, both sharing a common independent variable (x-axis) and a common dependent variable (y-axis). Both datasets to contain the same number of points, regions should overlap.

- Fourier transform both datasets

```
• # Applying FFT on both datasets
•     fftdataset1 = fft(dataset1)
•     fftdataset2 = fft(dataset2)
```

- Perform an inverse Fourier transform on the content of the first four data points within the transformed data set. The vectors returned shall be labelled as dataset1_dc and dataset2_dc.

```
•     dataset1_dc = fftdataset1
•     dataset2_dc = fftdataset2
•     dataset1_dc[0:4] = ifft(dataset1_dc[0:4])
•     dataset2_dc[0:4] = ifft(dataset2_dc[0:4])
```

- Calculate the sum of the values, S, of the independent variable, starting at the fifth data point and continuing to the end of the data set. Determine the '40%' location. This is the data point, at which the sum of the values of the independent variable, starting at the fifth data point and continuing to the data point, is greater than or equal to 40% of the value calculated.

```
• def find_point(array):
•     total_sum = sum(array)
•     target_sum = 0.4 * total_sum
•     current_sum = 0
•
•     for i, num in enumerate(array):
•         current_sum += num
•         if current_sum >= target_sum:
•             return i
```

```
fortyper_dataset1 = find_point(fftdataset1[4:])
fortyper_dataset2 = find_point(fftdataset2[4:])
```

- Determine the ‘break-point’ location. This is five data points higher than the minimum of the two ‘40% location’ determined in Step above.

```

• forty_percent_point=min(fortypercent_dataset1,fourtyper_dataset2)
• breakpoint_location = forty_percent_point + 5

```

- Copy both transformed datasets into a variable named dataset1_lo and dataset2_lo respectively.

```

dataset1_lo = fftdataset1
dataset2_lo = fftdataset2

```

- Apply the filter values as shown in code snippets to both datasets to generate a filtered transformed working dataset and perform an inverse transform on both datasets. Lo_dataset1 and Lo_dataset2 are thus obtained.

```

•   ## Lo dataset Computation
•   dataset1_lo = fftdataset1
•   dataset2_lo = fftdataset2
•
•   dataset1_lo[breakpoint_location-3] = 1
•   dataset1_lo[breakpoint_location-2] = 0.834
•   dataset1_lo[breakpoint_location-1] = 0.667
•   dataset1_lo[breakpoint_location] = 0.500
•   dataset1_lo[breakpoint_location+1] = 0.334
•   dataset1_lo[breakpoint_location+2] = 0.167
•   dataset1_lo[breakpoint_location+3] = 0
•
•   dataset2_lo[breakpoint_location-3] = 1
•   dataset2_lo[breakpoint_location-2] = 0.834
•   dataset2_lo[breakpoint_location-1] = 0.667
•   dataset2_lo[breakpoint_location] = 0.500
•   dataset2_lo[breakpoint_location+1] = 0.334
•   dataset2_lo[breakpoint_location+2] = 0.167
•   dataset2_lo[breakpoint_location+3] = 0
•
•   dataset1_lo = ifft(dataset1_lo)
•   dataset2_lo = ifft(dataset2_lo)

```

- Apply the filter values as shown in code snippets to both datasets to generate a filtered transformed working dataset and perform an inverse transform on both datasets. Hi_dataset1 and Hi_dataset2 are thus obtained.

```

•   ## Hi dataset Computation
•   dataset1_hi = fftdataset1
•   dataset2_hi = fftdataset2
•
•   dataset1_hi[breakpoint_location-3] = 0
•   dataset1_hi[breakpoint_location-2] = 0.167
•   dataset1_hi[breakpoint_location-1] = 0.334
•   dataset1_hi[breakpoint_location] = 0.500
•   dataset1_hi[breakpoint_location+1] = 0.667
•   dataset1_hi[breakpoint_location+2] = 0.834
•   dataset1_hi[breakpoint_location+3] = 1
•
•   dataset2_hi[breakpoint_location-3] = 0
•   dataset2_hi[breakpoint_location-2] = 0.167
•   dataset2_hi[breakpoint_location-1] = 0.334
•   dataset2_hi[breakpoint_location] = 0.500
•   dataset2_hi[breakpoint_location+1] = 0.667
•   dataset2_hi[breakpoint_location+2] = 0.834
•   dataset2_hi[breakpoint_location+3] = 1
•
•   dataset1_hi = ifft(dataset1_hi)
•   dataset2_hi = ifft(dataset2_hi)

```

- Amplitude Difference Measure to be calculated using:

$$ADM(n) = \left| \frac{\alpha}{\beta} \right| + \left| \frac{\chi}{\delta} \right| \exp \left\{ \left| \frac{\chi}{\delta} \right| \right\}$$

$$\alpha = (|Lo_1(n)| - |Lo_2(n)|)$$

$$\beta = \frac{1}{N} \sum_{i=1}^N (|Lo_1(i)| + |Lo_2(i)|)$$

Where =>

$$\chi = (|DC_1(n)| - |DC_2(n)|)$$

$$\delta = \frac{1}{N} \sum_{i=1}^N (|DC_1(i)| + |DC_2(i)|)$$

- `## Amplitude Difference Measure (ADM) Calculation`
- `ADMi_new = np.zeros(N)`
- `alpha_ADMi_new = np.abs(np.abs(dataset1_lo)-np.abs(dataset2_lo))`
- `beta_ADMi_new = (1 / N) * np.sum(np.abs(dataset1_lo) + np.abs(dataset2_lo))`
- `X_ADMi_new = np.abs(np.abs(dataset1_dc)-np.abs(dataset2_dc))`
- `delta_ADMi_new = (1 / N) * np.sum(np.abs(dataset1_dc) + np.abs(dataset2_dc))`
- `ADMi_new = (alpha_ADMi_new/beta_ADMi_new) + (X_ADMi_new/delta_ADMi_new)*np.exp(X_ADMi_new/delta_ADMi_new)`
- `ADMi_new[ADMi_new > 5e1] = 0`
- `### Points having ADM > 50 are ignored during computation of ADM (Numerical Error)`

- Calculate the mean value of ADM :

- `ADM_new = np.sum(ADMi_new)/N`

- Calculate the ADM confidence histogram. Range of values can be divided into 6 categories as shown:

Table.1

FSV value (quantitative)	FSV interpretation (qualitative)
Less than 0.1	Excellent
Between 0.1 and 0.2	Very good
Between 0.2 and 0.4	Good
Between 0.4 and 0.8	Fair
Between 0.8 and 1.6	Poor
Greater than 1.6	Very poor

- `## ADM Histogram`
- `bins_ADM_new = [0, 0.1, 0.2, 0.4, 0.8, 1.6, np.inf]`
- `labels_ADM_new = ['Excellent', 'Very Good', 'Good', 'Fair', 'Poor', 'Very Poor']`
- `hist_ADM_new, bins_ADM_new = np.histogram(ADMi_new, bins=bins_ADM_new)`
- `total_samples_ADM_new = len(ADMi_new)`
- `proportions_ADM_new = hist_ADM_new / total_samples_ADM_new`

- Calculate Derivatives in preparation for FDM calculation.

The first order derivatives of the Lo and Hi datasets are to be calculated.

The second order derivatives of the Hi datasets are to be calculated. The first and second derivatives are currently obtained by a simple difference approach $Lo'(i) = Lo(i+1) - Lo(i-1)$. At the edges of the domain is used a correction:

for left edge of the domain:

$$\frac{dLo}{dx}(i) = \frac{Lo(i+1) - Lo(i)}{\Delta}$$

for right edge of the domain:

$$\frac{dLo}{dx}(i) = \frac{Lo(i) - Lo(i-1)}{\Delta}.$$

For all internal points:

$$\frac{dLo}{dx}(i) = \frac{Lo(i+1) - Lo(i-1)}{\Delta}$$

where Δ is set to 1 in all cases.

```
# Feature Difference Measure (FDM) Calculation
# Requires 1st order derivatives for Lo and Hi Datasets.
# Requires 2nd order derivative for Hi Datasets.

# CALCULATING LO' (First Order Derivative for Lo Datasets)
dataset1_lo_firstder = np.zeros(N)
dataset2_lo_firstder = np.zeros(N)

dataset1_lo_firstder[0] = dataset1_lo[1]-dataset1_lo[0]
for i in range(1,N-1):
    dataset1_lo_firstder[i] = dataset1_lo[i+1]-dataset1_lo[i-1]

dataset1_lo_firstder[N-1] = dataset1_lo[N-1]-dataset1_lo[N-2]
dataset2_lo_firstder[0] = dataset2_lo[1]-dataset2_lo[0]
for i in range(1,N-1):
    dataset2_lo_firstder[i] = dataset2_lo[i+1]-dataset2_lo[i-1]

dataset2_lo_firstder[N-1] = dataset2_lo[N-1]-dataset2_lo[N-2]
```

```

# CALCULATING Hi' (First Order Derivative for Hi Datasets)

dataset1_hi_firstder = np.zeros(N)
dataset2_hi_firstder = np.zeros(N)

dataset1_hi_firstder[0] = dataset1_hi[1]-dataset1_hi[0]

for i in range(1,N-1):
    dataset1_hi_firstder[i] = dataset1_hi[i+1]-dataset1_hi[i-1]

dataset1_hi_firstder[N-1] = dataset1_hi[N-1]-dataset1_hi[N-2]
dataset2_hi_firstder[0] = dataset2_hi[1]-dataset2_hi[0]

for i in range(1,N-1):
    dataset2_hi_firstder[i] = dataset2_hi[i+1]-dataset2_hi[i-1]

dataset2_hi_firstder[N-1] = dataset2_hi[N-1]-dataset2_hi[N-2]

# CALCULATING Hi'' (Second Order Derivative for Hi Datasets)

dataset1_hi_secondder = np.zeros(N)
dataset2_hi_secondder = np.zeros(N)

dataset1_hi_secondder[0] = dataset1_hi_firstder[1]-dataset1_hi_firstder[0]

for i in range(1,N-1):
    dataset1_hi_secondder[i] = dataset1_hi_firstder[i+1]-
dataset1_hi_firstder[i-1]

    dataset1_hi_secondder[N-1] = dataset1_hi_firstder[N-1]-
dataset1_hi_firstder[N-2]

dataset2_hi_secondder[0] = dataset2_hi_firstder[1]-dataset2_hi_firstder[0]

for i in range(1,N-1):
    dataset2_hi_secondder[i] = dataset2_hi_firstder[i+1]-
dataset2_hi_firstder[i-1]

    dataset2_hi_secondder[N-1] = dataset2_hi_firstder[N-1]-
dataset2_hi_firstder[N-2]

```


- Calculating point-by-point FDM.

$$\text{FDM}_1(f) = \frac{|Lo_1'(f)| - |Lo_2'(f)|}{\frac{2}{N} \sum_{i=1}^N (|Lo_1'(i)| + |Lo_2'(i)|)}$$

$$\text{FDM}_2(f) = \frac{|Hi_1'(f)| - |Hi_2'(f)|}{\frac{6}{N} \sum_{i=1}^N (|Hi_1'(i)| + |Hi_2'(i)|)}$$

$$\text{FDM}_3(f) = \frac{|Hi_1''(f)| - |Hi_2''(f)|}{\frac{7.2}{N} \sum_{i=1}^N (|Hi_1''(i)| + |Hi_2''(i)|)}$$

```
FDM1 = np.zeros(N)
FDM2 = np.zeros(N)
FDM3 = np.zeros(N)
FDM1_denom = (2 / N) * np.sum(np.abs(dataset1_lo_firstder) +
np.abs(dataset2_lo_firstder))
FDM1 =
np.subtract(np.abs(dataset1_lo_firstder),np.abs(dataset2_lo_firstder))
FDM1 = FDM1/FDM1_denom
FDM2_denom = (6 / N) * np.sum(np.abs(dataset1_hi_firstder) +
np.abs(dataset2_hi_firstder))
FDM2 =
np.subtract(np.abs(dataset1_hi_firstder),np.abs(dataset2_hi_firstder))
FDM2 = FDM2/FDM2_denom
FDM3_denom = (7.2 / N) * np.sum(np.abs(dataset1_hi_secondder) +
np.abs(dataset2_hi_secondder))
FDM3 =
np.subtract(np.abs(dataset1_hi_secondder),np.abs(dataset2_hi_secondder))
FDM3 = FDM3/FDM3_denom
```

Now, calculate the point value of FDM , then a single value (mean) for FDM.

$$\text{FDM}(f) = 2(|\text{FDM}_1(f) + \text{FDM}_2(f) + \text{FDM}_3(f)|)$$

```
FDMi = 2*np.abs((FDM1 + FDM2 + FDM3))
FDM = np.sum(FDMi)/N
```

- Calculate the FDM confidence histogram as done for ADM.

```
• ## FDM Confidence Interval Histogram
• bins_FDM = [0, 0.1, 0.2, 0.4, 0.8, 1.6, np.inf]
• labels_FDM = ['Excellent', 'Very Good', 'Good', 'Fair', 'Poor', 'Very
Poor']
• hist_FDM, bins_FDM = np.histogram(FDMi, bins=bins_FDM)
```

```

• total_samples_FDM = len(FDMi)
• proportions_FDM = hist_FDM / total_samples_FDM

```

- Implementation of Grade and Spread into GDM calculation.

Instead of treating ADM and FDM as equally important, the representative nature of FSV has been improved by weighting the ADM and FDM in the GDM calculation based on the following :

- How concentrated the histogram bars are at one extreme or the other (Grade)
- How dispersed the histogram bars are around the mean value (Spread)

The GRADE value is computed by taking the number of classes, starting from the best (Excellent) to the worst (Very Poor), which include a User defined amount (named “threshold” and set at 85 % by default) of the total samples of the data sets to be compared. GRADE value ranges from 1 (best quality) to 6 (worst quality). The GRADE range is given by considering the range of the classes included in the GRADE value computation.

The SPREAD is computed by taking the number of classes, starting from the most populated to the lowest one, which includes a User defined amount (named “threshold” and set at 85 % by default) of the total samples of the data sets to be compared. SPREAD ranges from 1 (best quality) to 6 (worst quality). The SPREAD range is given by considering the range of the classes included in the SPREAD value computation.

Now, Computing the weights of K_{ADM} & K_{FDM} :

The algorithm is >

```

If SpreadADM < SpreadFDM
then
    KADM = 1
    KFDM = SpreadADM / SpreadFDM
Else if SpreadADM > SpreadFDM
Then
    KFDM = 1
    KADM = SpreadFDM / SpreadADM
Else
    KADM = 1
    KFDM = 1

```

```

# Implementing Grade & Spread
# Spread ADM
cumulative_data_ADM = np.cumsum(hist_ADM_new)
total_temp = cumulative_data_ADM[-1]
cumulative_percentage_ADM = cumulative_data_ADM / total_temp
# Calculate Grade
grade_ADM = np.sum(cumulative_percentage_ADM <= 0.85) + 1 # +1 because
np.sum starts from 0 index
# Calculate Spread
spread_ADM = np.argmax(cumulative_percentage_ADM >= 0.85) + 1 # +1 to
convert from index to count
# Spread FDM
cumulative_data_FDM = np.cumsum(hist_FDM)
total_temp2 = cumulative_data_FDM[-1]
cumulative_percentage_FDM = cumulative_data_FDM / total_temp2
# Calculate Grade
grade_FDM = np.sum(cumulative_percentage_FDM <= 0.85) + 1 # +1 because
np.sum starts from 0 index
# Calculate Spread
spread_FDM = np.argmax(cumulative_percentage_FDM >= 0.85) + 1 # +1 to
convert from index to count
## Calculating K_adm & k_fdm
if(spread_ADM < spread_FDM):
    k_adm = 1
    k_fdm = spread_ADM/spread_FDM
elif(spread_ADM > spread_FDM):
    k_fdm = 1
    k_adm = spread_FDM/spread_ADM
else:
    k_adm = 1
    k_fdm = 1

```

- Computation of Global Difference Measure (GDM)

$$GDM = \sum_{f_{\min}}^{f_{\max}} \sqrt{k_{ADM_i_cm} \cdot (ADM_i(x))^2 + k_{FDM_i_cm} \cdot (FDM_i(x))^2}$$

```

GDMi_new = np.zeros(N)
GDMi_new = np.sqrt(k_adm*np.square(ADMi_new)+k_fdm*np.square(FDMi))

```

- Calculate the Confidence histogram for GDM

```

## FDM Confidence Interval Histogram
bins_GDM = [0, 0.1, 0.2, 0.4, 0.8, 1.6, np.inf]
labels_GDM = ['Excellent', 'Very Good', 'Good', 'Fair', 'Poor', 'Very
Poor']
hist_GDM, bins_GDM = np.histogram(GDMi, bins=bins_GDM)
total_samples_GDM = len(GDMi)

```

- `proportions_GDM = hist_GDM / total_samples_GDM`

- Plotting the Confidence histograms for the ADM, FDM and GDM and returning the single values of ADM, FDM , GDM
The values can be interpreted using Table.1 and a Qualitative Natural Language comparison can be given to the datasets.

```
plt.figure(figsize=(6, 6))
plt.bar(labels_ADM_new, proportions_ADM_new, color='blue', alpha=0.7)
plt.xlabel('Confidence Level')
plt.ylabel('ADM proportion')
plt.title('ADM Confidence Histogram (modified)')
plt.figure(figsize=(6, 6))
plt.bar(labels_FDM, proportions_FDM, color='blue', alpha=0.7)
plt.xlabel('Confidence Level')
plt.ylabel('FDM proportion')
plt.title('FDM Confidence Histogram')
plt.figure(figsize=(6, 6))
plt.bar(labels_GDM_new, proportions_GDM_new, color='blue', alpha=0.7)
plt.xlabel('Confidence Level')
plt.ylabel('GDM proportion')
plt.title('GDM Confidence Histogram (Modified)')
plt.show()
return ADM_new, FDM, GDM_new
•
•
```

- Calling the Function

```
def main():
    Y = np.load('Y_henninger.npy')[40:] # (10, p)
    Y_predicted = np.load("Y_pred_henninger.npy") # (10, p)

    # Example usage:
    number_dataset = 9
    ADM_new, FDM, GDM_new = apply_fsv(
        Y[number_dataset],
        Y_predicted[number_dataset]
    )

if __name__ == "__main__":
    main()
```

(number_dataset points to which datasets are to be compared [0:9], specific to Jan He's data).

Example

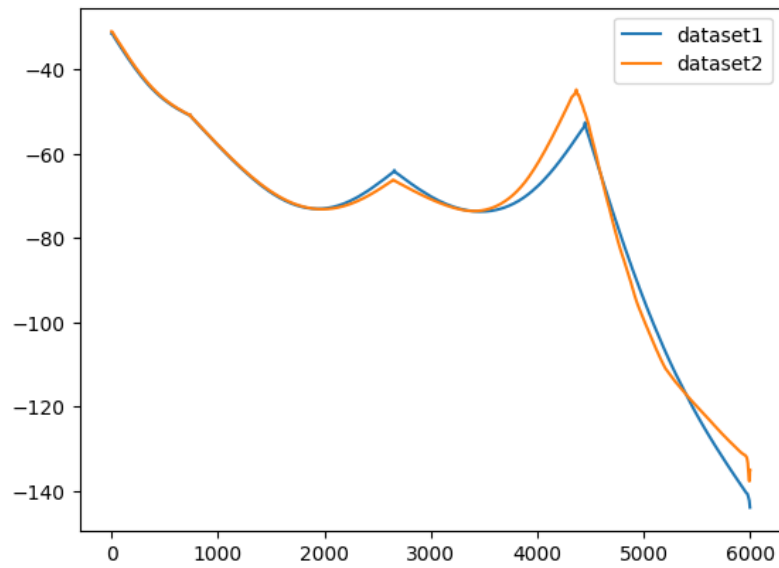
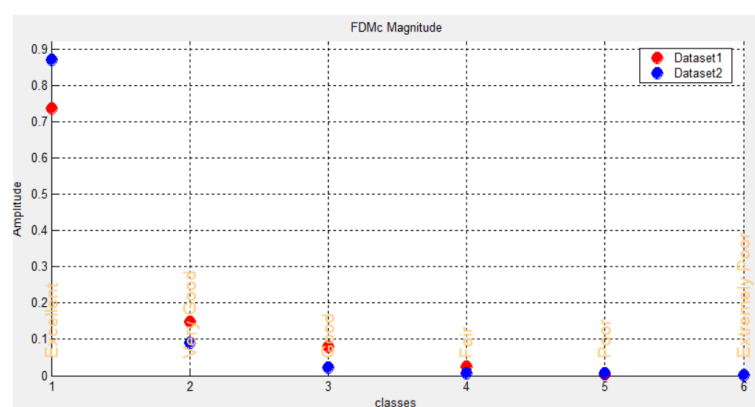
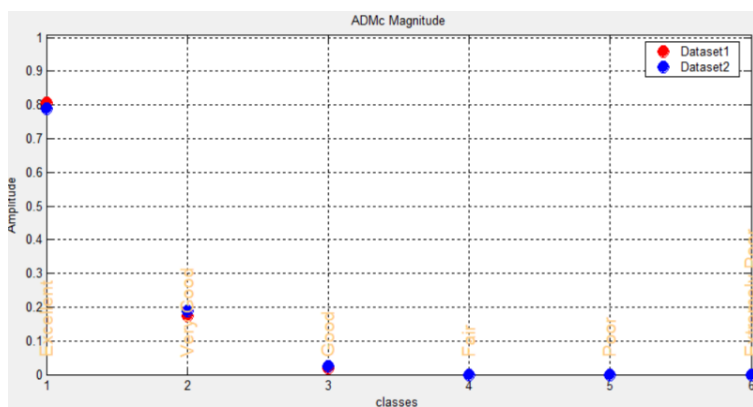
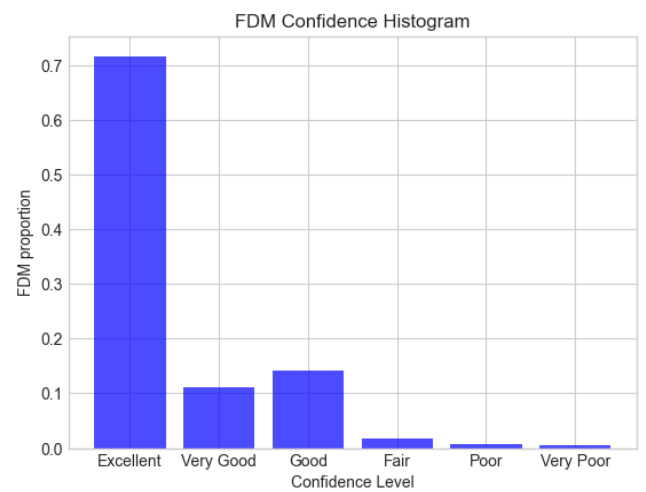
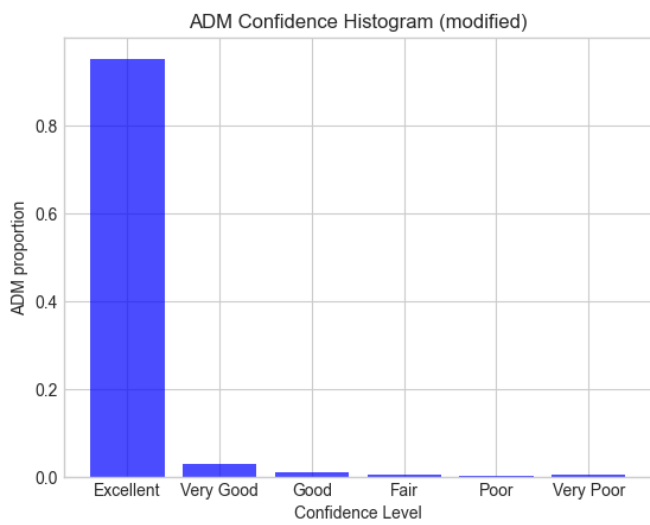
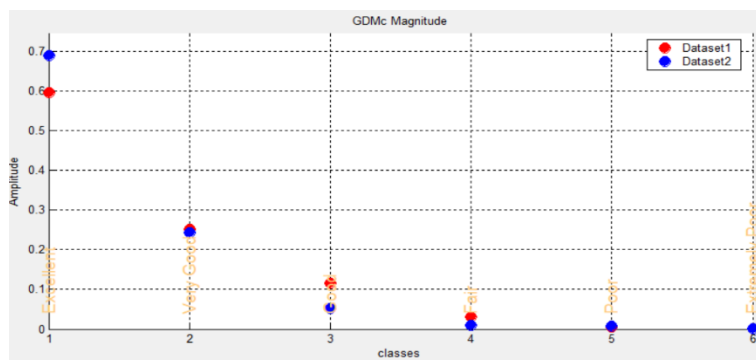
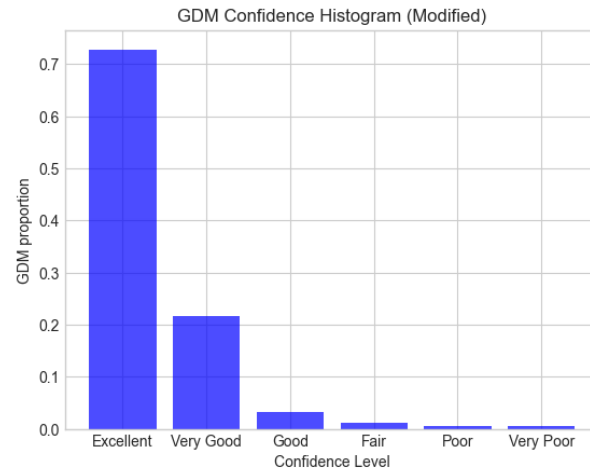


Fig.1 Two datasets being compared (6000 points)



ADM and FDM obtained from the Python Code (above) and the FSV Tool respectively(below)



GDM obtained from the Python Code (above) and the FSV Tool respectively(below)

The Single Value of the differences obtained are:

ADM: 0.079 - EXCELLENT

FDM: 0.111 – VERY GOOD

GDM: 0.121 – VERY GOOD

- From ADM Confidence Interval, we can see that the across the range of points, the trend mostly comes under excellent, hence matches almost identically
- From FDM Confidence Interval, some points spread out into Very good and good intervals; this inference is making sense as visually it could be seen that at one of the resonance points, the curves differ
- From the GDM single value, we see that the match is Very Good.

Further Scope of work

- Exploring more about FSV, found some interesting papers (fuzzy FSV?)
- Histogram Density – more bins could be used (check references)
- Finding a fix to the numerical errors occurring in ADM calculations
 - Current workaround is to not consider the values having ADM values $>5e1$ (this is a value fixed upon running multiple test runs, could be improved to a better value).
 - This doesn't alter the Confidence Intervals by much, as the values that are ignored are very few (0.16% of the total points). But it significantly improves the **one-value** of ADM, GDM thus obtained.
- Improving Feature Difference Measure (maybe resonances need to be accounted for manually)

References

- 1) IEEE Standard for Validation of Computational Electromagnetics Computer Modeling and Simulations (**IEEE Std 1597.1™-2008**)
- 2) A. P. Duffy, A. J. M. Martin, A. Orlandi, G. Antonini, T. M. Benson and M. S. Woolfson, "Feature selective validation (FSV) for validation of computational electromagnetics (CEM). part I-the FSV method," in IEEE Transactions on Electromagnetic Compatibility, vol. 48, no. 3, pp. 449-459, Aug. 2006, doi: 10.1109/TEMC.2006.879358.
- 3) V. Rajamani, C. F. Bunting, A. Orlandi and A. Duffy, "Introduction to feature selective validation (FSV)," 2006 IEEE Antennas and Propagation Society International Symposium, Albuquerque, NM, USA, 2006, pp. 601-604, doi: 10.1109/APS.2006.1710595.
- 4) D. Di Febo, F. de Paulis, A. P. Duffy and A. Orlandi, "Histogram density for the Feature Selective Validation (FSV) method," 9th IET International Conference on Computation in Electromagnetics (CEM 2014), London, UK, 2014, pp. 1-2, doi: 10.1049/cp.2014.0206.
- 5) hgs, H. G. Sasse. "Applying the Feature Selective Validation (FSV) method to quantifying rf measurement comparisons." (2009).
- 6) http://ing.univaq.it/uaqemc/FSV_4_0_3L/1DFSV_Manual.pdf
(***1D FSV Tool User's Guide Version: 4***)